Playwright Core: Setup, Locators & First Tests

Session 2 · Playwright Automation Bootcamp

TESTMART | LOCATORS | AUTO-WAIT | FIRST JOURNEY

What You Will Be Able to Do

01

Install & Configure

Set up Playwright for TestMart with correct baseURL

02

Understand Lifecycle

Explain browser, context, and page hierarchy

03

Master Locators

Use role, label, text, testid, CSS, and XPath correctly

04

Handle Edge Cases

Strict mode violations, auto-wait, spinner pattern, negative assertions

05

Full Journey

Complete Login → Search → Logout independently

Install and Configure for TestMart

Scaffold Playwright and point it at TestMart in three steps:

1

Scaffold

Run `npm init playwright@latest` — choose TypeScript, `tests/` folder, install browsers

2

Configure

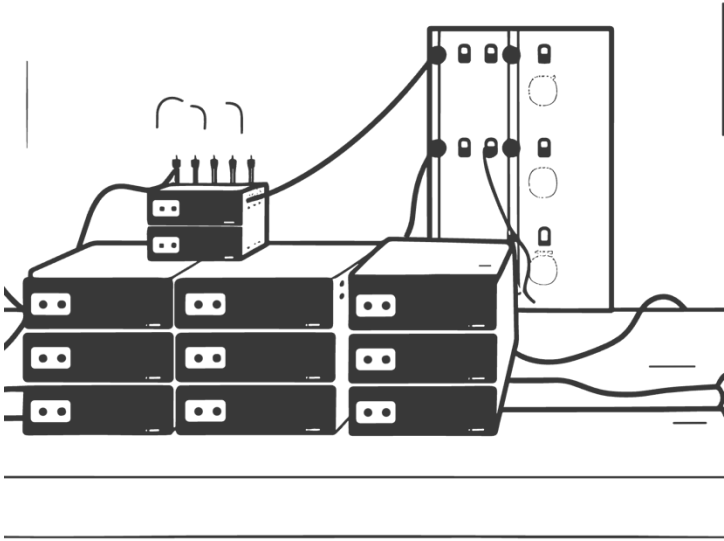
Set `baseURL: 'http://localhost:3000'` in `playwright.config.ts`. All `page.goto('/')` calls resolve relative to this URL.

3

Verify

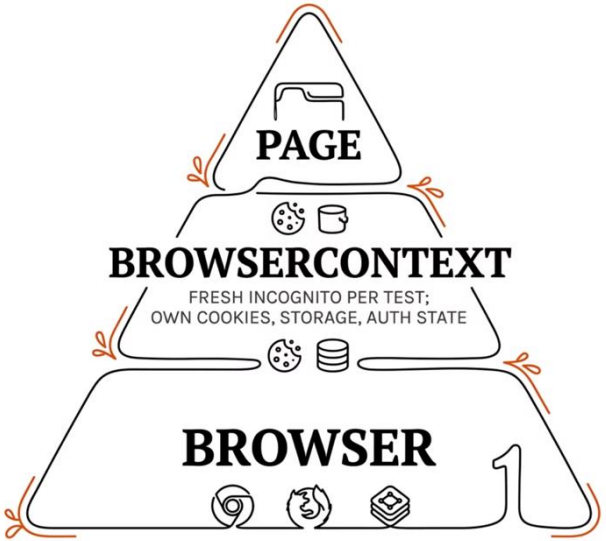
Run `npx playwright test` — should print **0 tests, no errors**

```
// playwright.config.ts
baseURL: 'http://localhost:3000',
screenshot: 'only-on-failure',
trace: 'on-first-retry'
```



Browser, Context & Page Lifecycle

Three layers of isolation — understanding them prevents flaky tests.



Fresh Context Per Test

No state leaks between tests by default

Multi-Tab Support

A context can hold multiple pages

Auto-Managed Fixture

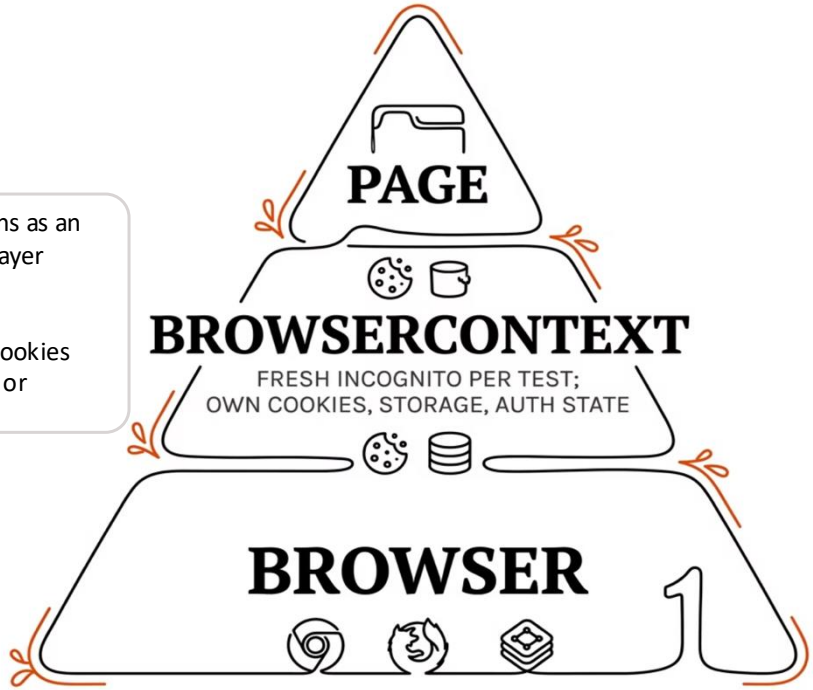
Playwright Test creates and tears down the `page` fixture automatically

⚠ If test A logs in as admin and test B shares the same context, test B is already logged in. Contexts prevent this.

Browser, Context & Page Lifecycle

Three layers of isolation — understanding them prevents flaky tests.

Playwright's `BrowserContext` functions as an independent, incognito-style session layer designed for rapid test isolation. This mechanism creates a fresh, isolated environment for each test. Covering cookies and storage, with near-zero overhead or memory impact.

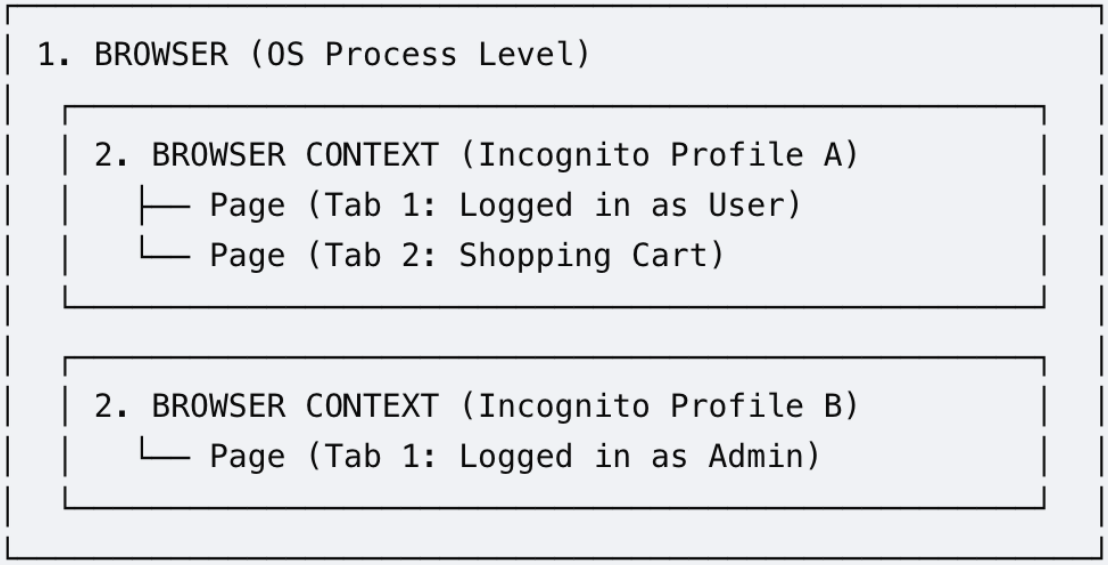


A Page in Playwright represents a single browser tab or window within a specific `BrowserContext`, providing isolated DOM execution for interactions like navigation and clicking. It features minimal overhead because pages within the same context share cookies and authentication, enabling multi-tab workflow simulation without re-login.

A Playwright browser instance represents a physical, high-overhead OS process such as Chromium or Firefox that handles low-level rendering and network tasks. To optimize performance, Playwright typically launches a single instance to be reused across multiple tests.

Browser, Context & Page Lifecycle

Three layers of isolation — understanding them prevents flaky tests.



Anatomy of a Playwright Test

Every test you write follows the same five-part structure. Here's the full breakdown.

```
import { test, expect } from '@playwright/test';

test('home page loads', async ({ page }) => {
  await page.goto('/');
  await expect(page.getByTestId('nav-logo')).toBeVisible();
});
```

1

Import

Pulls in the test runner and assertion library

2

Test Definition

Names the test — appears in HTML report and terminal

3

Fixture `{ page }`

Fresh browser tab per test — torn down automatically

4

Navigation

`await page.goto('/')` — relative to `baseURL` in config

5

Assertion

Auto-retries up to 5 seconds before failing

Keyword	Purpose
`import { test, expect }`	Test runner + assertion library
`test('name', async ...)`	Defines one test case
`{ page }`	Fresh tab, created & destroyed per test
`await`	Pauses until browser finishes — never skip
`expect(...).toBeVisible()`	Assertion with auto-retry

⚠ **Golden Rule:** Forgetting `await` means Playwright moves on before the browser acts. Always await every browser call.

Built-in Fixtures

A **fixture** is an object Playwright creates, configures, and tears down automatically per test. Declare what you need as parameters — Playwright handles the lifecycle.

Fixture	Type	When to use
`page`	Browser tab	Default for every UI test — a fresh tab per test
`context`	Isolated session	Set cookies, storage, or permissions before navigating
`browser`	Browser process	Rarely used directly — Playwright manages it
`request`	HTTP client	API preconditioning and hybrid tests (covered in Session 4)

```
// Page + context — inject cookies before navigating
test('pre-auth test', async ({ page, context }) => {
  await context.addCookies([{ name: 'token', value: '...', domain: 'localhost', path: '/' }])
  await page.goto('/dashboard')
})

// Page + request — create via API, verify in UI
test('create via API, verify in UI', async ({ page, request }) => {
  await request.post('/api/products', { data: { name: 'Widget', price: 9.99 } })
  await page.goto('/products')
  await expect(page.getByText('Widget')).toBeVisible()
})
```

Note: `request` is listed here for completeness. It will be used extensively starting from **Session 4 (Hybrid Testing)**.

Locator Strategy Priority

Choose the most resilient locator — CSS classes break on redesigns; role + name matches how screen readers see the page.

Rank	Strategy	Example	Best For
1st	Role (ARIA)	`getByRole('button', { name: 'Log in' })`	Buttons, links, inputs, headings
2nd	Label	`getByLabel('Email address')`	Form inputs with visible label
3rd	Placeholder	`getByPlaceholder('Search products...')`	Inputs without a visible label
4th	Test ID	`getByTestId('login-submit')`	Custom components, no semantic role
5th	Text	`getByText('Welcome back')`	Static visible copy
❌	CSS / XPath	`locator('.btn-primary')`	Last resort only

ⓘ **Why order matters:** `data-testid` attributes are explicit test contracts developers don't rename. Role + name is both accessible and resilient.

getByRole — The Most Powerful Locator

Matches what users actually see. Using the wrong role causes a `TimeoutError` — Playwright never matches.

HTML → ARIA Role Mapping

```
// button element
getByRole('button', { name: 'Search' })

// a href element
getByRole('link', { name: 'Browse Products' })

// h1–h6 elements
getByRole('heading', { name: 'Products' })

// input type="checkbox"
getByRole('checkbox', { name: 'Accept terms' })

// input type="text"
getByRole('textbox', { name: 'Search' })
```

Critical Distinction on TestMart

⚠ Mixing up `button` vs `link` role causes a `TimeoutError`. Always check the HTML element type first.

getByLabel, getByText & getByTestId

The other three strategies — each has a clear use case.

getByLabel — Form Inputs

```
getByLabel('Email address')
  .fill('tester@example.com')
getByLabel('Password').fill('Password123!')
```

getByText — Static Copy

```
expect(page.getByText(
  'Quality tools for quality engineers'
)).toBeVisible()
// partial match
expect(page.getByText('Mechanical')).toBeVisible()
```

Avoid `getByText` for interactive elements — prefer `getByRole` for buttons and links.

getByTestId — Custom Components

```
expect(page.getByTestId('loading-spinner'))
  .toBeVisible()
expect(page.getByTestId('cart-count'))
  .toHaveText('3')
```

1. **getByText**
   UX copy can change
2. **getByLabel**
   Accessibility contract
3. **getByTestId**
   Explicit test contract — most stable

CSS & XPath Locators

LAST RESORT, NOT FIRST CHOICE

When Justified

- Element has no ARIA role, label, or `data-testid`
- Navigating complex DOM relationships
- Third-party component you cannot modify

When to Avoid

- `.btn-primary` breaks when design system renames the class
- `//button[text()="Log in"]` breaks when label changes
- `getByRole('button', { name: 'Log in' })` is clearer and more resilient

By Element

```
page.locator('button')
```

By Attribute

```
page.locator('[data-testid="search-input"]')
```

XPath Text

```
//button[text()="Log in"]
```

XPath Contains

```
//*[contains(text(), "Keyboard")]
```

Strict Mode & Nested Locators

Playwright throws a strict mode violation when a locator matches more than one element.

The Problem

The Fix — Nested Locator

Or Use .first()

Prefer .filter()

Scopes by content — more intentional and readable

Use .first() Sparingly

Only when position is semantically meaningful

Avoid nth()

Index-based selectors are fragile — break on UI reorder

Nested Locators & Strict Mode

ONE ELEMENT, NOT TWELVE

Strict mode: Playwright refuses to act on a locator matching more than one element — prevents tests from silently acting on the wrong element.

❌ Wrong — 12 cards match

```
page.getByTestId('product-card')
  .getByTestId('add-to-cart-btn')
  .click() // error!
```

✅ Correct — scope first

```
const firstCard = page
  .getByTestId('product-card').first()
firstCard
  .getByTestId('add-to-cart-btn').click()

const thirdCard = page
  .getByTestId('product-card').nth(2)
```

ⓘ Strict mode is a **feature, not a bug**. If your locator matches two elements when you expected one, your test has a bug.

Performing Actions on Locators

You found the element — now act on it. The pattern: **locate, then act.**

```
const emailInput    = page.getByTestId('login-email')
const passwordInput = page.getByTestId('login-password')
const loginButton   = page.getByTestId('login-submit')

await emailInput.fill('standard_user@example.com')
await passwordInput.fill('Password123!')
await loginButton.click()
```

Action	Usage	Example
`fill(value)`	Type into an input	`emailInput.fill('user@example.com')`
`click()`	Click a button or link	`loginButton.click()`
`check()`	Tick a checkbox	`page.getByLabel('Accept terms').check()`
`hover()`	Move mouse over element	`page.getByTestId('product-card').first().hover()`
`press(key)`	Press a keyboard key	`searchInput.press('Enter')`
`selectOption(value)`	Choose a dropdown option	`page.getByLabel('Country').selectOption('AU')`
`textContent()`	Read element text	`const name = await nameEl.textContent()`
`inputValue()`	Read input value	`const val = await emailInput.inputValue()`

Key rule: Every action **auto-waits** — the same retry loop as assertions. `fill()` waits for visible + editable. `click()` waits for visible, stable, and enabled. No explicit waits needed.

Auto-Wait & Why `waitForTimeout` Is Wrong

Auto-Wait

Playwright waits for elements to be visible, enabled, and stable before acting — no manual waits needed

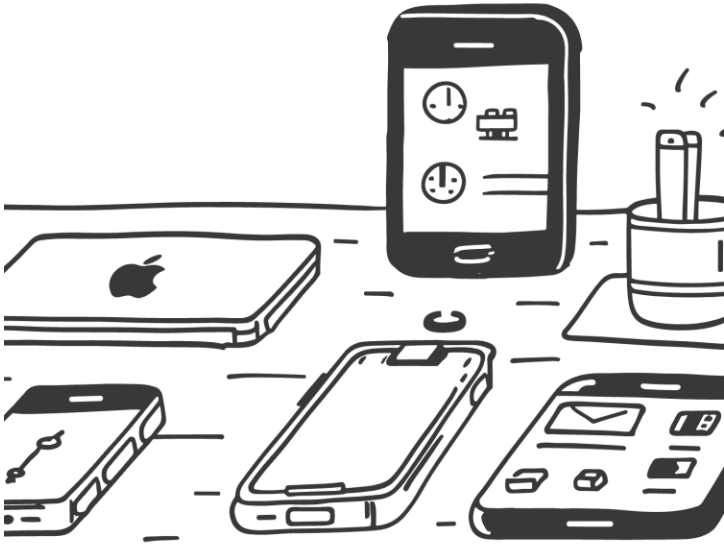
Spinner Pattern

Assert spinner is visible, then assert it's gone, then assert content — reliable for async-loaded data

Negative Assertions

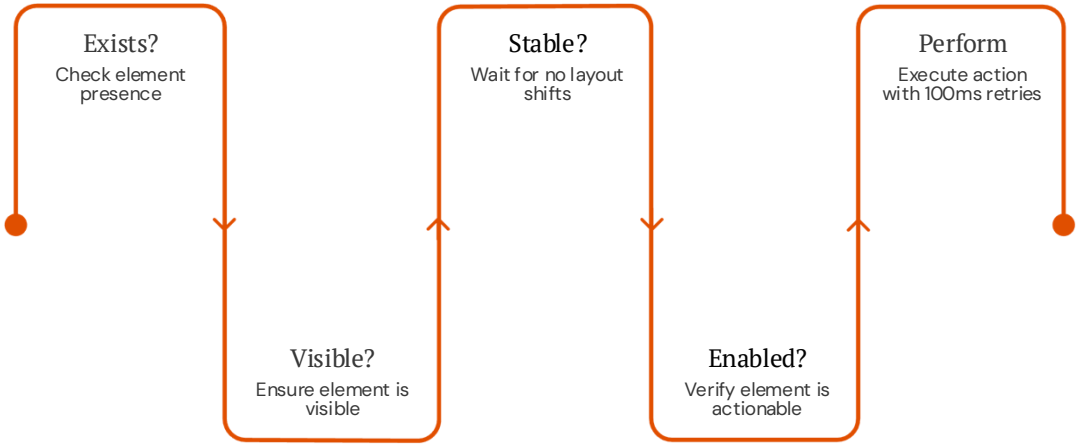
Use `not.toBeVisible()` to confirm elements disappear after an action

`waitForTimeout(2000)` is a hard sleep — it makes tests slow and flaky. Always use auto-wait or the spinner pattern instead.



Auto-Wait: How the Retry Loop Works

PLAYWRIGHT RETRIES SO YOU DONT HAVE TO



Actionability Checks

- `click()` — visible, stable, enabled, not obscured
- `fill()` — visible, enabled
- `check()` — visible, enabled

Default Timeouts

- Per action: **30 seconds**
- Per assertion: **5 seconds**

In Selenium you write `WebDriverWait` before every click. In Playwright, **that wait IS the click** — built in.